9. Adapter Design Pattern in Java

9.1 Introduction

The **Adapter Design Pattern** is a **structural pattern** that allows incompatible interfaces to work together by acting as a bridge.

9.2 Problem it Solves

When integrating a new component or library that doesn't match your system's expected interface, the Adapter pattern helps adapt it without modifying existing code.

9.3 What is Adapter Pattern?

Adapter wraps an existing class with a new interface so that it becomes compatible with client code expecting a different interface.

9.4 Real-World Analogy

A power adapter converts a three-prong plug into a two-prong outlet. Similarly, the Adapter pattern converts one interface into another.

9.5 Types of Adapters

5.1 Class Adapter (Using Inheritance)

- Adapts one interface to another by extending the adaptee class and implementing the target interface.
- Limited in Java due to single inheritance.

5.2 Object Adapter (Using Composition)

- Holds a reference to the adaptee object and implements the target interface.
- More flexible and preferred in Java.

9.6 Basic Structure

```
package com.mannemlokesh;

//Target Interface
interface Target {
	void request();
}

//Adaptee
class Adaptee {
	void specificRequest() {
		System.out.println("Called specificRequest()");
	}
}

//Adapter using composition
public class Adapter implements Target {
	private Adaptee adaptee = new Adaptee();
```

```
	public void request() {
		adaptee.specificRequest();
	}
}
```

9.7 Step-by-Step Examples

Example1: Class Adapter Example (using inheritance)

```
package com.mannemlokesh.example1;

interface Target {
	void request();
}

class Adaptee {
	void specificRequest() {
		System.out.println("Called specificRequest() via Class Adapter");
	}
}

class ClassAdapter extends Adaptee implements Target {
	public void request() {
		specificRequest();
	}
}

public class Example1ClassAdapterDemo {
	public static void main(String[] args) {
		Target target = new ClassAdapter();
		target.request();
	}
}
```

Output:

```
Called specificRequest() via Class Adapter
```

Example2: Object Adapter Example (using composition)

```
package com.mannemlokesh.example2;

interface Target {
	void request();
}

class Adaptee {
	void specificRequest() {
		System.out.println("Called specificRequest() via Object Adapter");
	}
}

class ObjectAdapter implements Target {
	private final Adaptee adaptee;

	public ObjectAdapter(Adaptee adaptee) {
		this.adaptee = adaptee;
	}
```

```
	public void request() {
		adaptee.specificRequest();
	}
}

public class Example2ObjectAdapterDemo {
	public static void main(String[] args) {
		Adaptee adaptee = new Adaptee();
		Target target = new ObjectAdapter(adaptee);
		target.request();
	}
}
```

Output:

```
Called specificRequest() via Object Adapter
```

Example3: Custom Reusable Object Manager

```
package com.mannemlokesh.example3;

interface MediaPlayer {
	void play(String audioType, String fileName);
}

interface AdvancedMediaPlayer {
	void playVlc(String fileName);

	void playMp4(String fileName);
}

class VlcPlayer implements AdvancedMediaPlayer {
	public void playVlc(String fileName) {
		System.out.println("Playing vlc file: " + fileName);
	}

	public void playMp4(String fileName) {
	}
}

class Mp4Player implements AdvancedMediaPlayer {
	public void playVlc(String fileName) {
	}

	public void playMp4(String fileName) {
		System.out.println("Playing mp4 file: " + fileName);
	}
}

class MediaAdapter implements MediaPlayer {
	AdvancedMediaPlayer advancedPlayer;

	public MediaAdapter(String audioType) {
		if (audioType.equalsIgnoreCase("vlc"))
			advancedPlayer = new VlcPlayer();
		else if (audioType.equalsIgnoreCase("mp4"))
			advancedPlayer = new Mp4Player();
	}
```

```
	public void play(String audioType, String fileName) {
		if (audioType.equalsIgnoreCase("vlc"))
			advancedPlayer.playVlc(fileName);
		else if (audioType.equalsIgnoreCase("mp4"))
			advancedPlayer.playMp4(fileName);
	}
}

class AudioPlayer implements MediaPlayer {
	MediaAdapter adapter;

	public void play(String audioType, String fileName) {
		if (audioType.equalsIgnoreCase("mp3")) {
			System.out.println("Playing mp3 file: " + fileName);
		} else if (audioType.equalsIgnoreCase("vlc") ||
					audioType.equalsIgnoreCase("mp4")) {
			adapter = new MediaAdapter(audioType);
			adapter.play(audioType, fileName);
		} else {
			System.out.println("Invalid format: " + audioType);
		}
	}
}

public class Example3MediaApp {
	public static void main(String[] args) {
		AudioPlayer player = new AudioPlayer();
		player.play("mp3", "song.mp3");
		player.play("mp4", "movie.mp4");
		player.play("vlc", "clip.vlc");
		player.play("avi", "unknown.avi");
	}
}
```

Output:

```
Playing mp3 file: song.mp3
Playing mp4 file: movie.mp4
Playing vlc file: clip.vlc
Invalid format: avi
```

Example4: Legacy Rectangle Drawing Adapter

```
package com.mannemlokesh.example4;

//Target
interface Shape {
	void draw(int x, int y, int width, int height);
}

//Adaptee
class LegacyRectangle {
	void draw(int x1, int y1, int x2, int y2) {
		System.out.println("Drawing from (" + x1 + ", " + y1 + ") to (" + x2
+ ", " + y2 + ")");
	}
}
```

```
//Adapter
class RectangleAdapter implements Shape {
	private final LegacyRectangle adaptee = new LegacyRectangle();

	public void draw(int x, int y, int width, int height) {
		adaptee.draw(x, y, x + width, y + height);
	}
}

public class Example4DrawingApp {
	public static void main(String[] args) {
		Shape shape = new RectangleAdapter();
		shape.draw(10, 20, 30, 40);
	}
}
```

Output:

```
Drawing from (10, 20) to (40, 60)
```

Example5: Third-Party Payment Gateway Integration

```
package com.mannemlokesh.example5;

//Target
interface PaymentProcessor {
	void pay(String customer, double amount);
}

//Adaptee (3rd-party library)
class StripeAPI {
	void makePayment(String userEmail, double dollars) {
		System.out.println("Stripe payment processed for " + userEmail +
                                                    " amount: " + dollars);
	}
}

//Adapter
class StripeAdapter implements PaymentProcessor {
	private StripeAPI stripe = new StripeAPI();

	public void pay(String customer, double amount) {
		stripe.makePayment(customer, amount);
	}
}

public class Example5PaymentApp {
	public static void main(String[] args) {
		PaymentProcessor processor = new StripeAdapter();
		processor.pay("alice@example.com", 99.99);
	}
}
```

Output:

```
Stripe payment processed for alice@example.com amount: 99.99
```

9.8 Adapter vs Other Patterns

Pattern	Purpose
Adapter	Match incompatible interfaces
Decorator	Add behavior to objects dynamically
Facade	Simplify a complex subsystem
Bridge	Separate abstraction from implementation

9.9 Pros and Cons

✅ **Pros**

- Promotes reusability and flexibility
- Allows integrating 3rd-party libraries
- Avoids changing existing client code

❌ **Cons**

- May introduce extra layer of indirection
- Complex if too many adapters are created

9.10 When to Use / Not to Use

Use When:

- You need to reuse an existing class that doesn't match your required interface
- You are integrating third-party APIs

Avoid When:

- You can modify the source code directly (not a library)
- Adapting causes high coupling

9.11 Summary

Feature	Description
Purpose	Convert one interface to another
Techniques	Inheritance (class) / Composition (object)
Use Cases	Legacy code, API bridges, 3rd-party tools

The **Adapter Pattern** is ideal when integrating incompatible systems or standardizing usage without modifying the original code.